

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

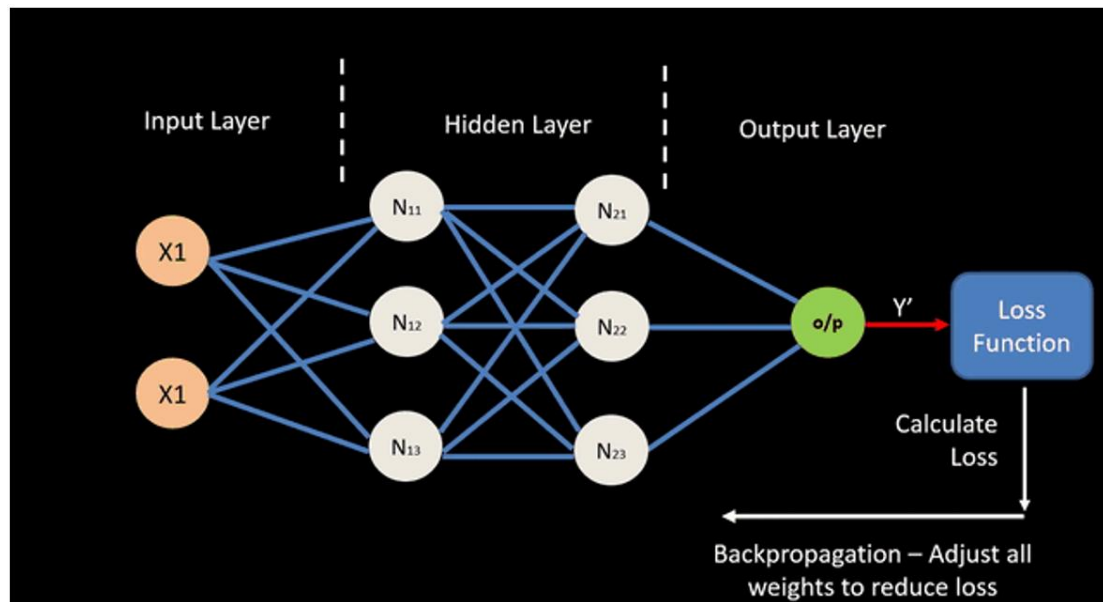
Анализ больших текстовых данных и текстовый поиск

Лекция 5

29 сентября 2025

Обучение нейронных сетей

Обучение нейронных сетей происходит методом **обратного распространения ошибки (backpropagation)**



Обучение нейронных сетей

- Нейронные сети обучаются с помощью различных модификаций градиентного спуска. Чтобы применять градиентный спуск, нужно уметь вычислять производную функции потерь по весам сети.
- Сам метод обратного распространения нужен именно для эффективного вычисления этих производных (градиентов), особенно когда сеть многослойная.
- При прямом проходе (forward pass) мы вычисляем все промежуточные значения (активации, взвешенные суммы и т.д.), и именно они используются в обратном проходе для вычисления градиентов.

Обучение нейронных сетей

Принцип работы — рекурсивное применение правила цепочки

Метод основан на рекурсивном применении правила для дифференцирования сложных функций (chain rule)

$$f(x) = g_m(g_{m-1}(\dots g_1(x) \dots))$$

$$\frac{\partial f}{\partial x} = \frac{\partial g_m}{\partial g_{m-1}} \cdot \frac{\partial g_{m-1}}{\partial g_{m-2}} \dots \frac{\partial g_2}{\partial g_1} \cdot \frac{\partial g_1}{\partial x}$$

Производная композиции функций равна произведению производных каждой функции по её аргументу, начиная с самой внешней и до самой внутренней

Обучение нейронных сетей

Рассмотрим на примере одного нейрона

x_1, x_2, x_n – входы в нейрон

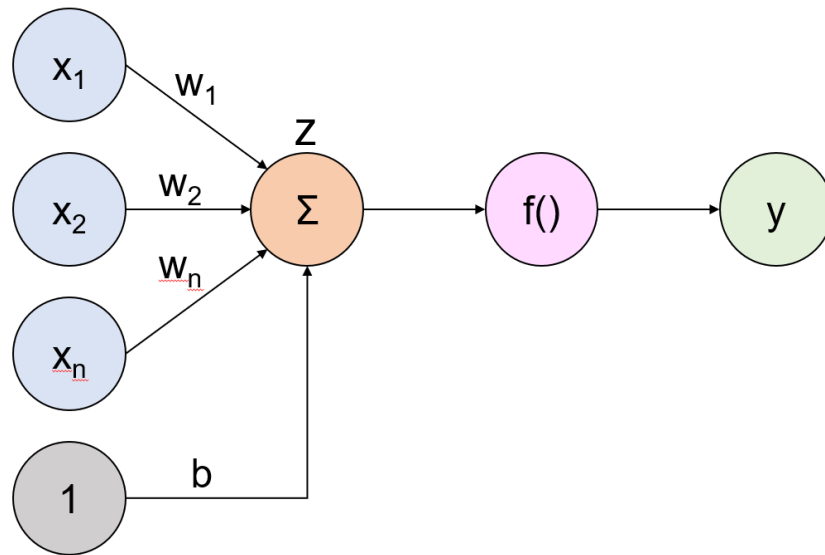
w_1, w_2, w_n – веса нейрона

Σ – сумматор

$f(x)$ – функция активации

y – выход нейрона

b – смещение



$$f(x_1w_1 + x_2w_2 + \dots + x_nw_n + b) = f\left(\sum_{i=1}^n x_iw_i + b\right)$$

Обучение нейронных сетей

$\hat{y}$ — правильный ответ (целевое значение)

Сумматор:

$$z = \sum_{i=1}^n w_i x_i + b$$

Предсказание нейрона:

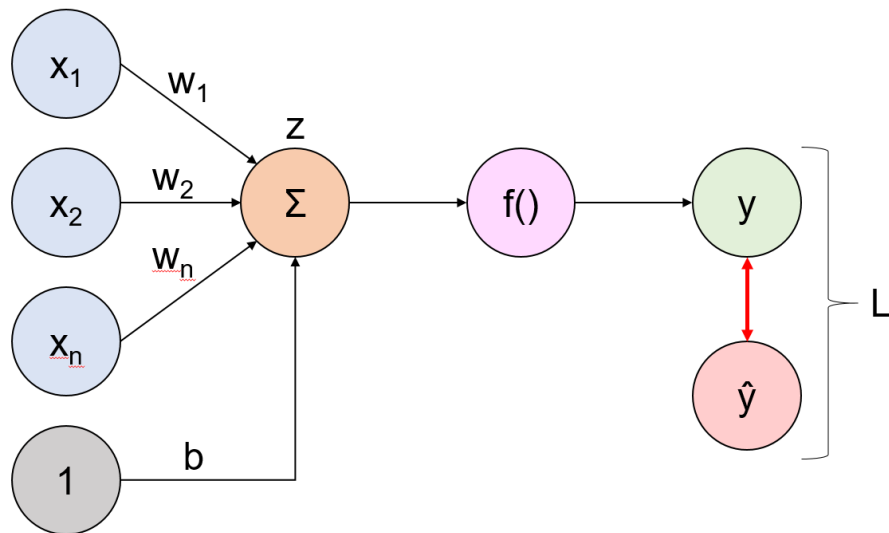
$$y = f(z)$$

Функция потерь MSE:

$$L = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial L}{\partial w_i} = ?$$

$$\frac{\partial L}{\partial b} = ?$$

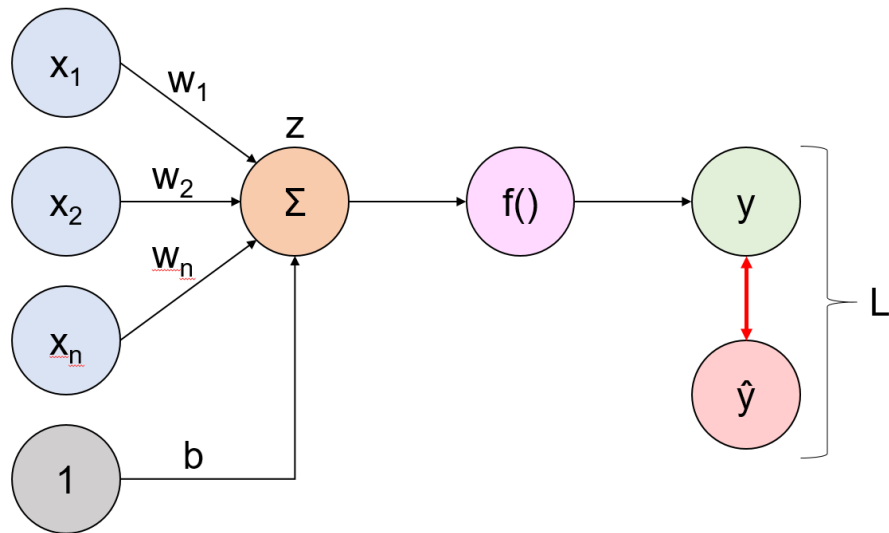


Обучение нейронных сетей

По правилу цепочки:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$$



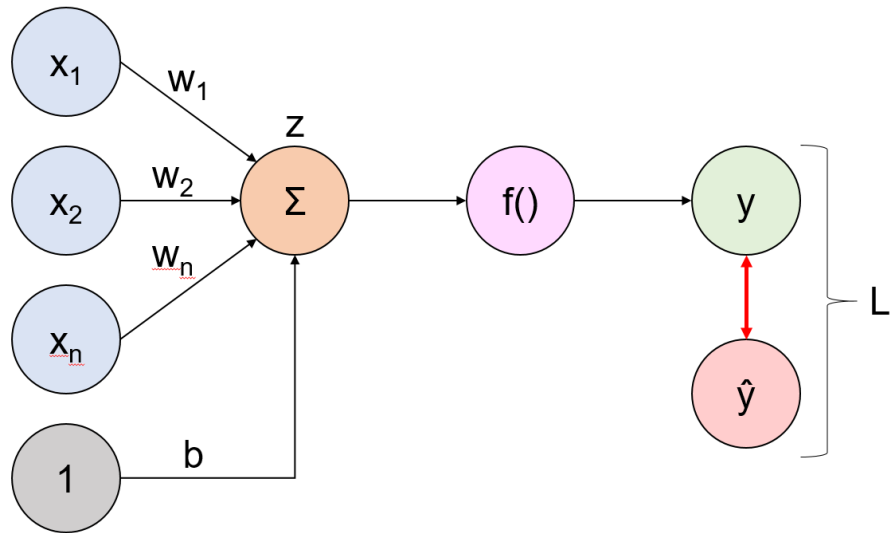
Обучение нейронных сетей

$$\frac{\partial L}{\partial y} = (y - \hat{y})$$

$$\frac{\partial y}{\partial z} = f'(z)$$

$$z = w_1x_1 + w_2x_2 + \dots + w_ix_i + \dots + w_nx_n + b$$

$$\frac{\partial z}{\partial w_i} = x_i \quad \frac{\partial z}{\partial b} = 1$$

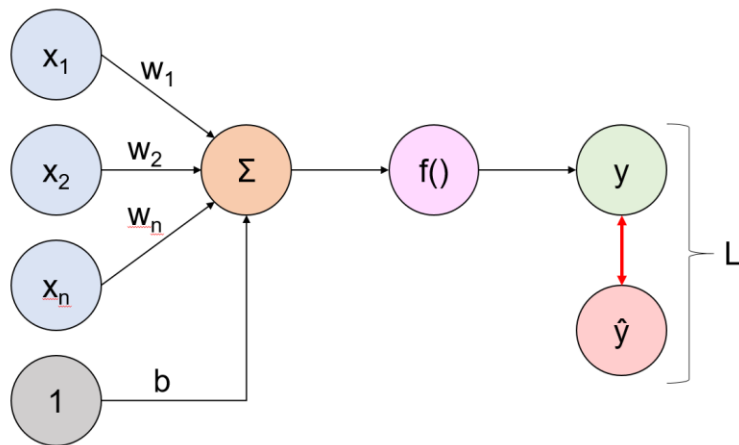


Обучение нейронных сетей

В результате получили градиенты:

$$\frac{\partial L}{\partial w_i} = (y - \hat{y}) \cdot f'(z) \cdot x_i$$

$$\frac{\partial L}{\partial b} = (y - \hat{y}) f'(z)$$



Далее можем сделать шаг обновления весов и смещения:

$$w_i^{(1)} = w_i^{(0)} - \eta \cdot \frac{\partial L}{\partial w_i}$$

$$b^{(1)} = b^{(0)} - \eta \cdot \frac{\partial L}{\partial b}$$

η — learning rate

Обучение нейронных сетей

Выполним расчет на примере

Значения весов, смещения и входных данных указаны на схеме

Функция активации – сигмоида:

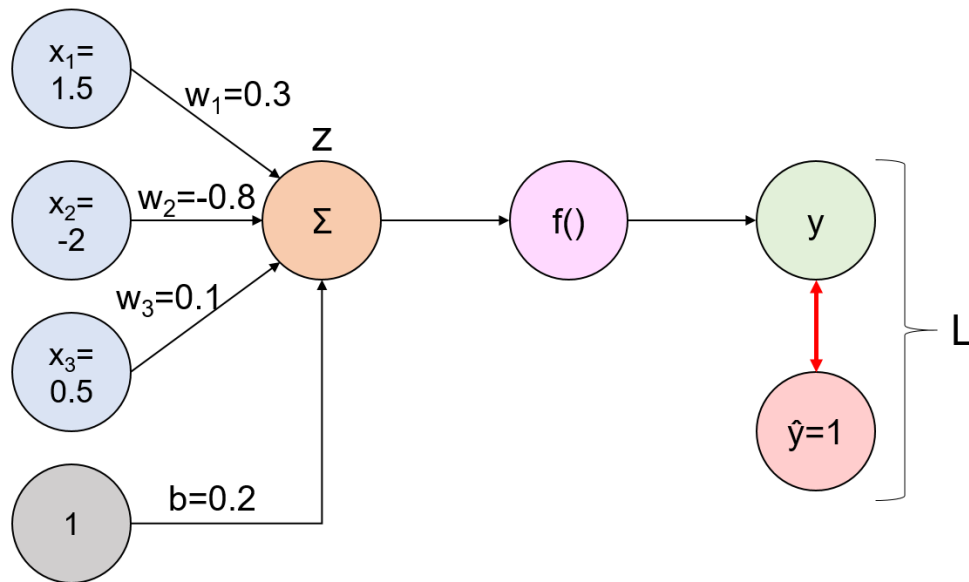
$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

Производная сигмоиды:

$$f'(z) = \sigma(z)(1 - \sigma(z))$$

Целевое значение $\hat{y} = 1$

**Нужно сделать шаг обновления
весов и смещения**



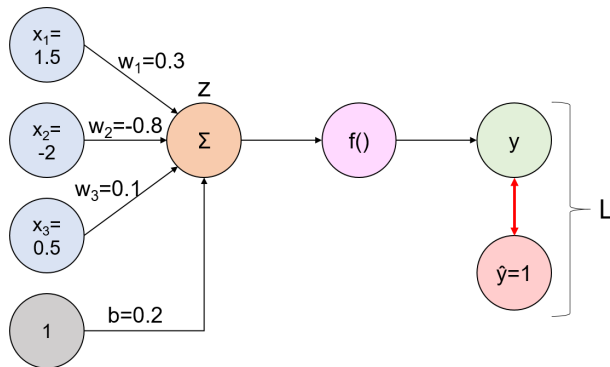
Обучение нейронных сетей

Forward Propagation: 

$$z = \sum_i w_i^{(0)} x_i + b^{(0)} = 0.3 \cdot 1.5 + (-0.8) \cdot (-2.0) + 0.1 \cdot 0.5 + 0.2 = 2.3$$

$$y = \sigma(z) = \frac{1}{1 + e^{-2.3}} \approx 0.908877$$

$$L = \frac{1}{2} (y - \hat{y})^2 = \frac{1}{2} (0.908877 - 1)^2 \approx 0.004151697$$



Обучение нейронных сетей

Backward Propagation:



Для весов:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i} = (y - \hat{y}) y(1 - y) x_i$$

$$x_1 = 1.5: \frac{\partial L}{\partial w_1} = (-0.0911229610) \cdot (0.082819567) \cdot (1.5) \approx -0.0113201463$$

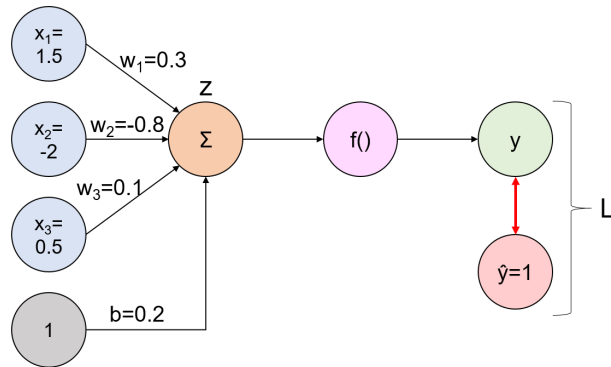
$$x_2 = -2.0: \frac{\partial L}{\partial w_2} = (-0.0911229610) \cdot (0.082819567) \cdot (-2.0) \approx +0.0150935283$$

$$x_3 = 0.5: \frac{\partial L}{\partial w_3} = (-0.0911229610) \cdot (0.082819567) \cdot (0.5) \approx -0.0037733821$$

Для смещения:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b} = (y - \hat{y}) y(1 - y) \cdot 1$$

$$\frac{\partial L}{\partial b} \approx (-0.0911229610) \cdot (0.082819567) \cdot 1 \approx -0.0075467642$$



Обучение нейронных сетей

Делаем шаг обновления весов и смещения:

$$w_i^{(1)} = w_i^{(0)} - \eta \cdot \frac{\partial L}{\partial w_i} \quad b^{(1)} = b^{(0)} - \eta \cdot \frac{\partial L}{\partial b}$$

$$w_1^{(1)} = 0.3 - 0.1 \cdot (-0.0113201463) = 0.301132015$$

$$w_2^{(1)} = -0.8 - 0.1 \cdot (+0.0150935283) = -0.801509353$$

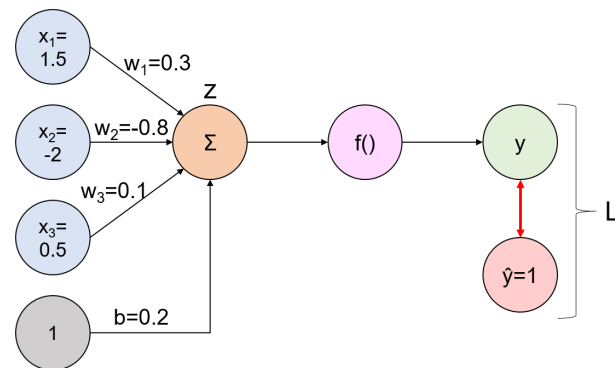
$$w_3^{(1)} = 0.1 - 0.1 \cdot (-0.0037733821) = 0.100377338$$

$$b^{(1)} = 0.2 - 0.1 \cdot (-0.0075467642) = 0.200754676$$

Проверка (ещё один forward propagation после шага):

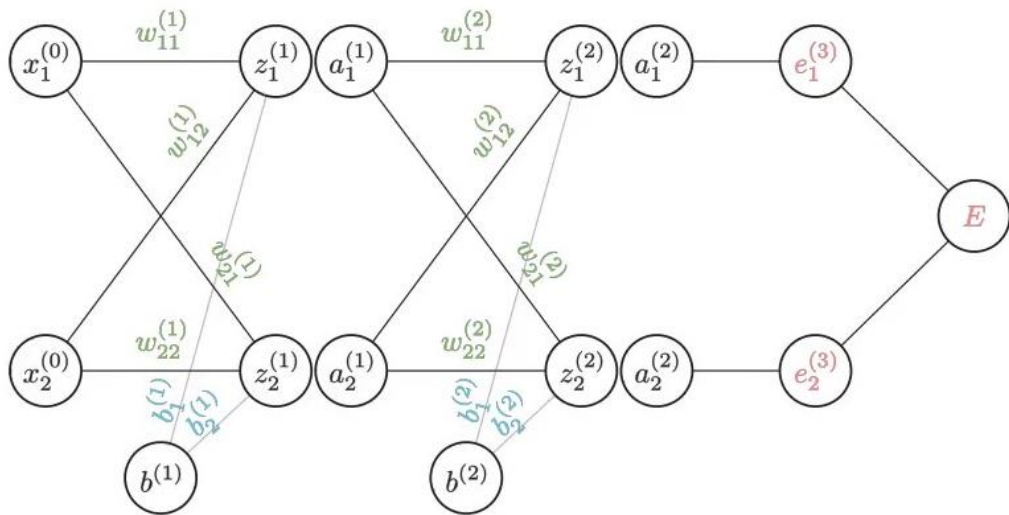
$$z^{(1)} = \sum_i w_i^{(1)} x_i + b^{(1)} \approx 2.305660073 \quad y^{(1)} = \sigma(z^{(1)}) \approx 0.909344720$$
$$L^{(1)} = \frac{1}{2} (y^{(1)} - \hat{y})^2 \approx 0.004109190$$

Ошибка (функция потерь) уменьшилась



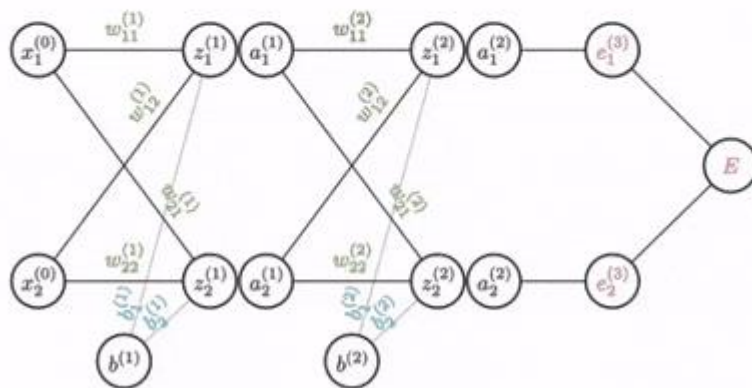
Обучение нейронных сетей

Рассмотрим полносвязную нейронную сеть с двумя входами и двумя выходами. z – сумматоры, a – функции активации, e – ошибки выходов, E – результирующая ошибок



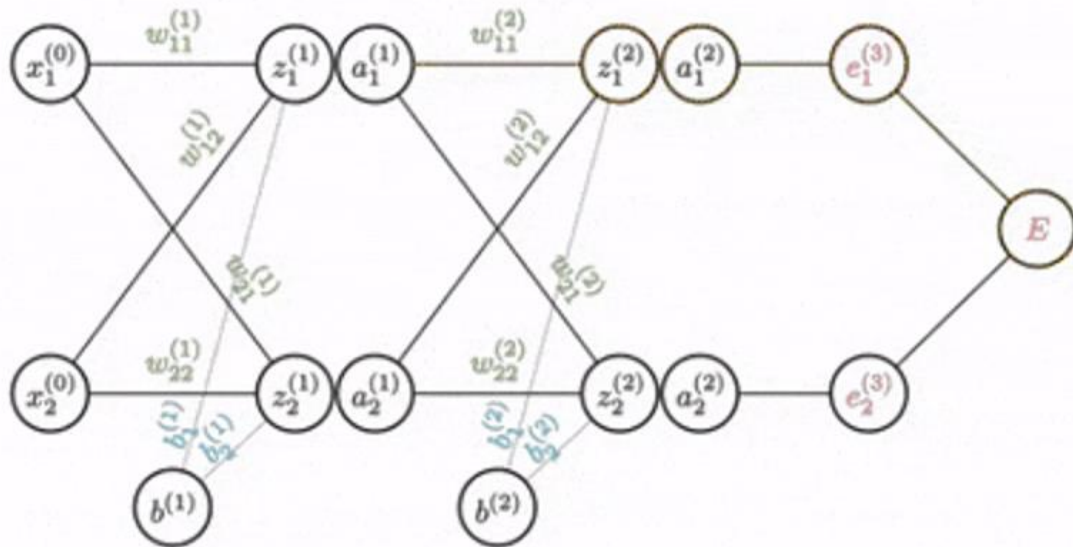
Обучение нейронных сетей

Конечной целью обратного распространения ошибки является нахождение изменения ошибки по отношению к весам сети. Мы начинаем с узла ошибки и движемся назад, шаг за шагом, беря частную производную текущего узла по узлу предыдущего слоя



Обучение нейронных сетей

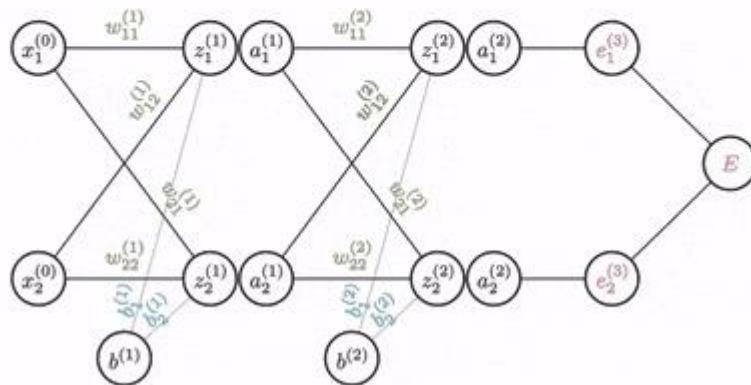
$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$$



Обучение нейронных сетей

Аналогично считается дифференциал ошибки по каждому весу

$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$$



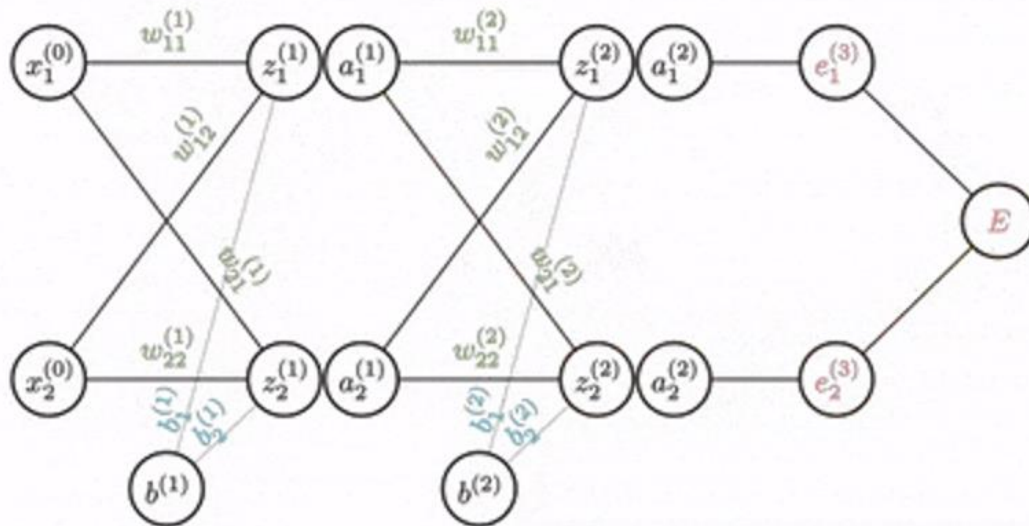
Обучение нейронных сетей

$$\frac{\partial E}{\partial w_{11}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$$

$$\frac{\partial E}{\partial w_{12}^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}}$$

$$\frac{\partial E}{\partial w_{21}^{(2)}} = \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}}$$

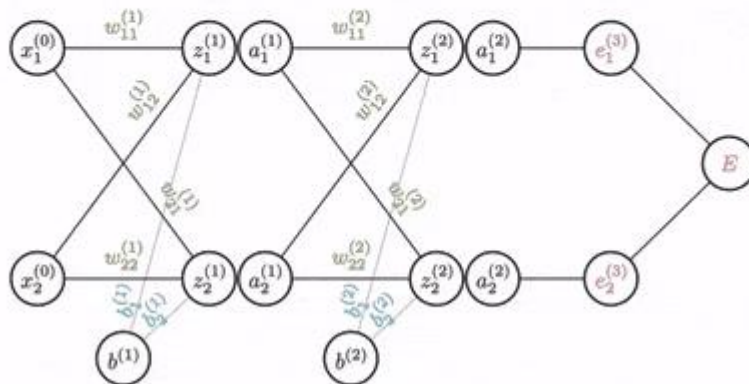
$$\frac{\partial E}{\partial w_{22}^{(2)}} = \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}}$$



Обучение нейронных сетей

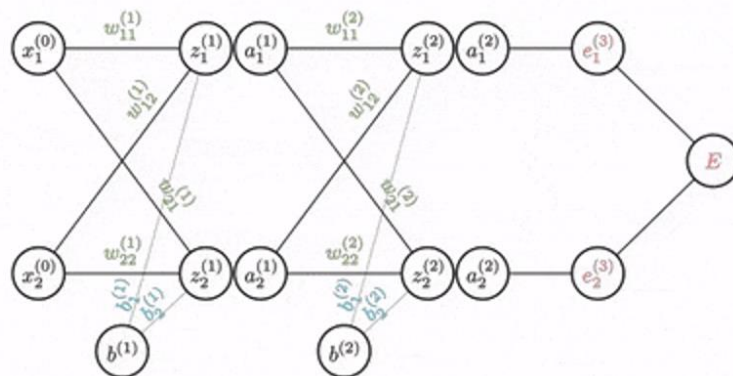
Приведение к матричному виду

$$\begin{aligned}\frac{\partial E}{\partial w_{11}^{(2)}} &= \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} & \frac{\partial E}{\partial w_{12}^{(2)}} &= \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(2)}} &= \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}} & \frac{\partial E}{\partial w_{22}^{(2)}} &= \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}}\end{aligned}$$



Обучение нейронных сетей

$$\begin{bmatrix} \frac{\partial E}{\partial w_{11}^{(2)}} & \frac{\partial E}{\partial w_{12}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(2)}} & \frac{\partial E}{\partial w_{22}^{(2)}} \end{bmatrix} = \begin{bmatrix} \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \\ \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \end{bmatrix} \odot \begin{bmatrix} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{bmatrix} \odot \begin{bmatrix} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} & \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}} \\ \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}} & \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}} \end{bmatrix}$$



$\odot$ — произведение Адамара (Hadamard product). Поэлементное умножение матриц: каждый элемент с индексами i, j — это произведение элементов с индексами i, j исходных матриц

Обучение нейронных сетей

$$\begin{bmatrix} \frac{\partial E}{\partial w_{11}^{(2)}} & \frac{\partial E}{\partial w_{12}^{(2)}} \\ \frac{\partial E}{\partial w_{21}^{(2)}} & \frac{\partial E}{\partial w_{22}^{(2)}} \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \\ \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \end{bmatrix} \odot \begin{bmatrix} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{bmatrix}}_{\delta^L} \odot \begin{bmatrix} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} & \frac{\partial z_1^{(2)}}{\partial w_{12}^{(2)}} \\ \frac{\partial z_2^{(2)}}{\partial w_{21}^{(2)}} & \frac{\partial z_2^{(2)}}{\partial w_{22}^{(2)}} \end{bmatrix}$$

$\frac{\partial E}{\partial W^{(2)}} \quad \quad \frac{\partial E}{\partial A^{(2)}} \quad \frac{\partial A^{(2)}}{\partial Z^{(2)}} \quad \quad \frac{\partial Z^{(2)}}{\partial W^{(2)}}$

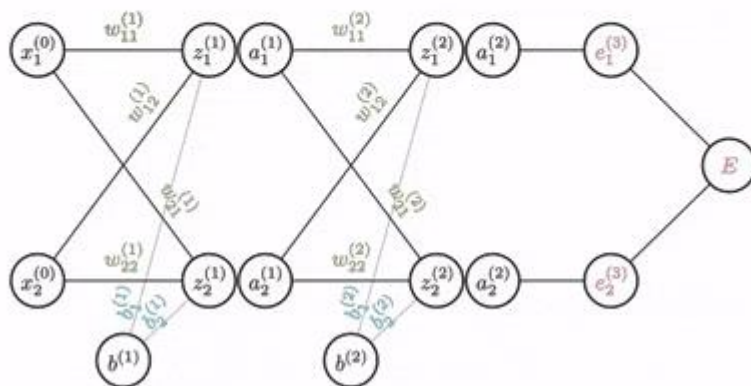
$$\frac{\partial E}{\partial W^{(2)}} = \delta^L \odot \frac{\partial Z^{(2)}}{\partial W^{(2)}}$$

Первая часть матричного произведения вынесена в отдельную переменную, потому что будет использоваться для расчета градиентов на более ранних слоях

Обучение нейронных сетей

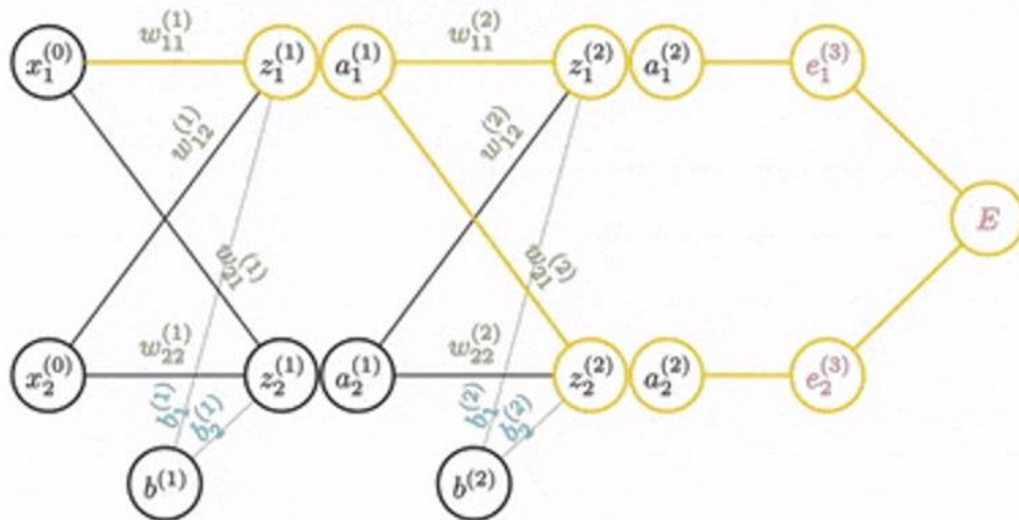
Для расчета градиента на более ранних слоях применяется тот же метод.

Теперь от узла полной ошибки к конкретному весу могут вести несколько путей распространения градиента. Когда градиенты из разных ветвей сходятся в одном нейроне, они суммируются, после чего продолжается дальнейшее применение правила цепочки.



Обучение нейронных сетей

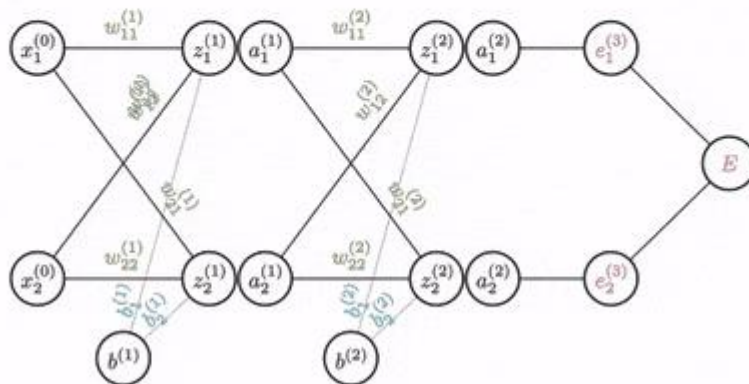
$$\frac{\partial E}{\partial w_{11}^{(1)}} = \left(\frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \frac{\partial e_2^{(3)}}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \right) \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$



Обучение нейронных сетей

Все остальные градиенты выводятся тем же методом

$$\frac{\partial E}{\partial w_{11}^{(1)}} = \left(\delta_1^{(1)} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \delta_2^{(1)} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \right) \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$



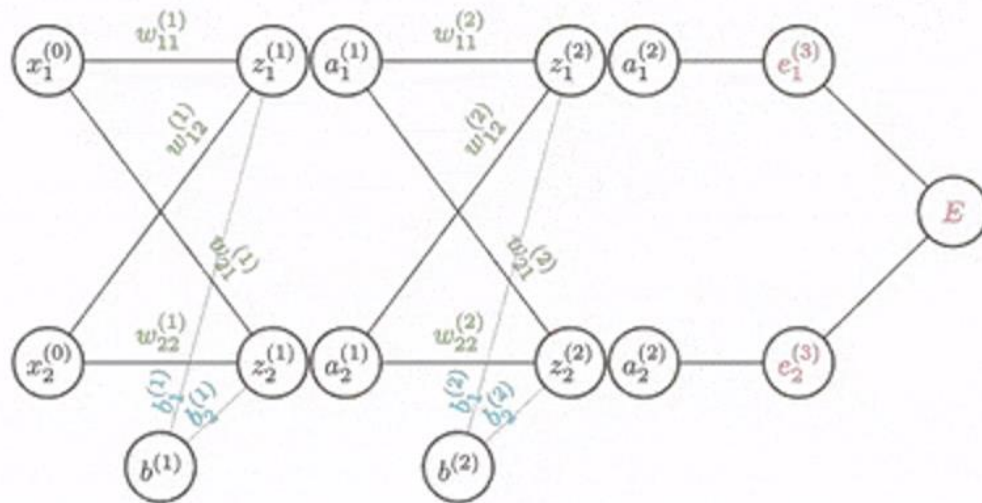
Обучение нейронных сетей

$$\frac{\partial E}{\partial w_{11}^{(1)}} = \left(\delta_1^L \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \delta_2^L \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \right) \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$

$$\frac{\partial E}{\partial w_{12}^{(1)}} = \left(\delta_1^L \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \delta_2^L \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \right) \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{12}^{(1)}}$$

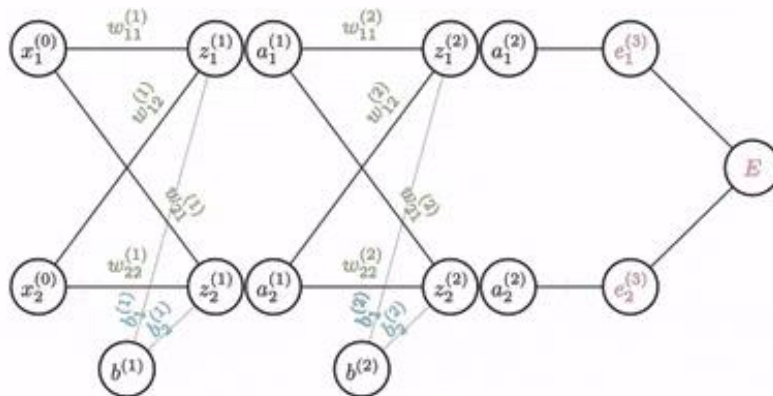
$$\frac{\partial E}{\partial w_{21}^{(1)}} = \left(\delta_1^L \frac{\partial z_1^{(2)}}{\partial a_2^{(1)}} + \delta_2^L \frac{\partial z_2^{(2)}}{\partial a_2^{(1)}} \right) \frac{\partial a_2^{(1)}}{\partial z_2^{(1)}} \frac{\partial z_2^{(1)}}{\partial w_{21}^{(1)}}$$

$$\frac{\partial E}{\partial w_{22}^{(1)}} = \left(\delta_1^L \frac{\partial z_1^{(2)}}{\partial a_2^{(1)}} + \delta_2^L \frac{\partial z_2^{(2)}}{\partial a_2^{(1)}} \right) \frac{\partial a_2^{(1)}}{\partial z_2^{(1)}} \frac{\partial z_2^{(1)}}{\partial w_{22}^{(1)}}$$



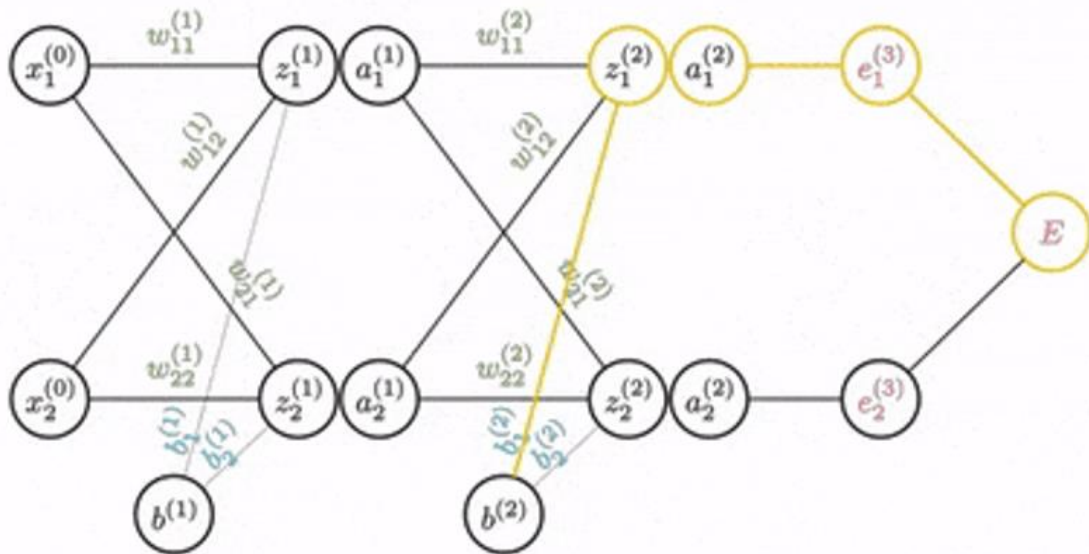
Обучение нейронных сетей

Градиент для смещения рассчитывается аналогично



Обучение нейронных сетей

$$\frac{\partial E}{\partial b_1^{(2)}} = \frac{\partial e_1^{(3)}}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial b_1^{(2)}}$$



Нормирование данных

Градиентный спуск сильно зависит от масштаба признаков. Если разные признаки измеряются в разных единицах (например, один в тысячах, другой в долях), то и производные функции потерь по соответствующим весам будут сильно различаться. В результате шаги обновления параметров окажутся несбалансированными: для одного признака вес может быстро достичь оптимума, для другого — почти не измениться, а для третьего — из-за слишком крупного шага «вылететь» в бесконечность и вызвать расходимость.

Поэтому при разнородных масштабах сложно подобрать единый learning rate, который обеспечит стабильную и быструю сходимость алгоритма.

Чтобы избежать этой проблемы, данные предварительно **нормируют** (например, с помощью StandardScaler), приводя все признаки к сопоставимому масштабу. Это делает поверхность функции потерь «более круглой», шаги градиентного спуска становятся равномерными по всем направлениям, и сеть обучается устойчивее и быстрее.

Нормирование данных

Есть матрица признаков $X \in \mathbb{R}^{m \times d}$

m — количество объектов (строк), d — количество признаков (столбцов)

Для каждого признака $j = 1, \dots, d$ считаем:

- Среднее по столбцу $\mu_j = \frac{1}{m} \sum_{i=1}^m X_{ij}$
- Дисперсию по столбцу $\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (X_{ij} - \mu_j)^2$
- Стандартное отклонение $\sigma_j = \sqrt{\sigma_j^2}$

Тогда стандартная нормировка каждого элемента столбца:

$$X'_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}$$

В результате каждый признак имеет среднее 0 и дисперсию 1

Инициализация весов

При инициализации весов в нейронной сети важно учитывать их масштаб. Если задать их просто случайным образом, могут возникнуть проблемы:

- слишком маленькие веса приводят к затуханию градиентов и замедляют обучение
- слишком большие веса вызывают взрыв градиентов и делают обучение нестабильным

Чтобы избежать этих эффектов, существуют специальные методы инициализации:

- **Xavier (Glorot)** инициализация — рассчитана в первую очередь для активаций sigmoid и tanh. Задаёт распределение весов с дисперсией, зависящей от числа входов и выходов нейрона, что помогает удерживать величины сигналов и градиентов в разумных пределах
- **He (Xe)** инициализация — оптимизирована для ReLU и её вариантов. Использует нормальное распределение с дисперсией, зависящей от количества входов, что позволяет лучше справляться с особенностями активации ReLU

Инициализация весов. Xavier

Xavier (или Glorot) инициализация — это метод задания начальных весов нейронной сети, предложенный Ксавье Глоро и Яном Лекуном в 2010 году. Его цель — предотвратить затухание или взрыв градиентов, чтобы обучение проходило стабильно и эффективно.

Идея метода: веса подбираются так, чтобы сохранять дисперсию как входных данных, так и градиентов при прохождении через слой. Благодаря этому нейросеть не теряет сигнал на больших глубинах и не сталкивается с резкими скачками при обновлении параметров.

Инициализация весов. Xavier

Для активаций с примерно симметричными функциями (например, $\tanh$, линейная функция) веса берутся из **равномерного распределения** с нулевым средним и границами:

$$w_{ij}^{(k)} \sim U \left[-\sqrt{\frac{6}{n_{in}(k) + n_{out}(k)}}, \sqrt{\frac{6}{n_{in}(k) + n_{out}(k)}} \right]$$

Для несимметричных функций активации (например, sigmoid , ReLU) используется **нормальное распределение**:

$$w_{ij}^{(k)} \sim \mathcal{N} \left(0, \frac{2}{n_{in}(k) + n_{out}(k)} \right)$$

где $n_{in}(k)$ — число входов в слой k , а $n_{out}(k)$ — число выходов из него

$U[a,b]$ — это равномерное распределение (Uniform distribution). Это значит, что вес выбирается случайно в пределах от a до b , и все значения из этого интервала равновероятны

$\mathcal{N}(\mu, \sigma^2)$ — это нормальное (гауссово) распределение (Normal distribution). Здесь μ — среднее значение (обычно 0), σ^2 — дисперсия. То есть большинство значений весов будет сосредоточено около μ , а вероятность убывает по мере удаления от него.

Инициализация весов. He

He-инициализация (или инициализация Хе) — это способ задания начальных весов нейронной сети, предложенный Kaiming He и соавторами. Метод специально разработан для слоёв с функцией активации из семейства ReLU, чтобы избежать проблем затухающих или взрывающихся градиентов.

Ключевая идея — подобрать распределение весов так, чтобы дисперсия сигналов на выходе слоя сохранялась на разумном уровне.

1. Нормальное распределение

Вес w задаётся как случайная величина, распределённая нормально с нулевым средним и дисперсией, зависящей от числа входов:

$$w \sim \mathcal{N}\left(0, \frac{2}{n}\right)$$

где n — количество входных связей нейрона.

Таким образом, значения весов сосредоточены около нуля, но достаточно «разбросаны», чтобы учесть специфику ReLU, которая обнуляет отрицательные входы

Инициализация весов. Не

2. Равномерное распределение

Вместо нормального распределения можно взять равномерное:

$$w \sim U[-a, a], \quad a = \sqrt{\frac{6}{n}}$$

где n — количество входов

Выбор между нормальным и равномерным распределением остаётся за исследователем: оба варианта применяются на практике, а окончательное решение обычно зависит от задачи и результатов экспериментов.

Взрыв градиентов (Exploding Gradient)

При обратном распространении, если множители (веса или производные) больше 1, то градиенты экспоненциально растут с каждым слоем.

Последствия

- Обновления весов становятся чрезмерно большими
- Loss скачет или становится NaN
- Веса уходят в бесконечность, обучение полностью рушится



Взрыв градиентов (Exploding Gradient)

Проявления

- Loss резко увеличивается, появляется нестабильность
- Значения весов и активаций становятся очень большими
- Иногда уже на первых эпохах всё превращается в NaN

Причины

- Слишком большие начальные веса
- Высокий learning rate
- Очень глубокая сеть без специальных приёмов стабилизации

Решения

- Gradient clipping (ограничение нормы или значения градиентов)
- Меньший learning rate
- Корректная инициализация весов (Xavier, He)
- Нормализация входных данных и активаций

Gradient clipping (ограничение градиентов)

Gradient clipping — это метод, который предотвращает взрыв градиентов при обучении нейросети. Идея проста: если норма (или значение) градиента слишком велика, мы ограничиваем её сверху до заданного порога

1. Clipping по норме (L2-norm clipping)

Вычисляем норму градиента:

$$\|g\|_2 = \sqrt{\sum_i g_i^2}$$

Если $\|g\|_2 > \text{threshold}$, то масштабируем: $g = g \cdot \frac{\text{threshold}}{\|g\|_2}$

То есть уменьшаем весь градиент пропорционально, чтобы его норма не превышала порог

2. Clipping по значению (value clipping)

Ограничиваем каждую компоненту градиента: $g_i \in [-\text{threshold}, \text{threshold}]$

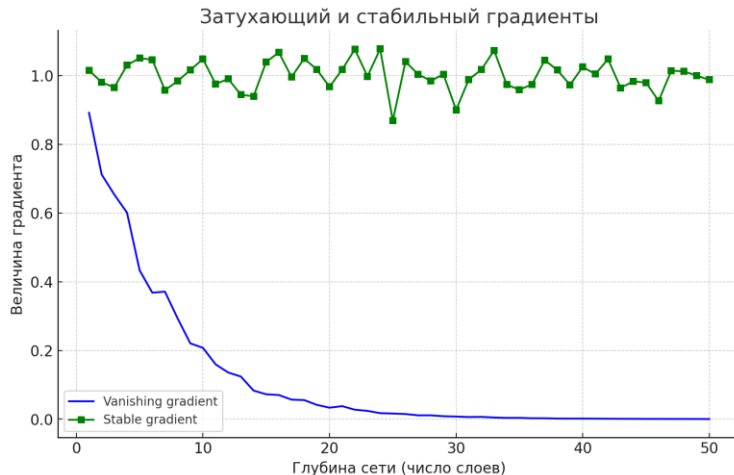
Первый способ используется чаще, так как сохраняет направление градиента

Затухание градиентов (Vanishing Gradient)

При обратном распространении ошибки градиенты вычисляются как произведение многих производных функций активации и весов. Если эти множители меньше 1, то при прохождении через десятки слоёв значения градиента экспоненциально уменьшаются

Последствия

- В глубоких слоях градиенты становятся практически нулевыми
- Веса этих слоёв почти не обновляются



Затухание градиентов (Vanishing Gradient)

Проявления

- Обучение очень медленное или вообще не начинается
- Loss застревает на высоких значениях и почти не снижается

Причины

- Использование функций активации с малыми производными (сигмоида, tanh)
- Большая глубина сети
- Неудачная инициализация весов

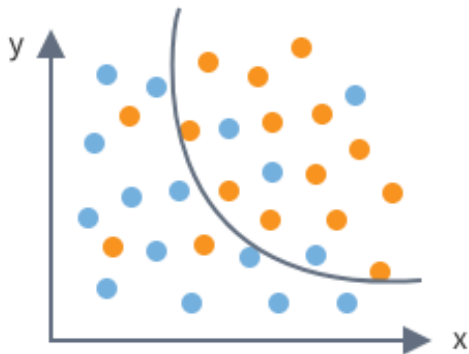
Решения

- Активации ReLU и её варианты (Leaky ReLU, ELU и др.)
- Xavier или He инициализация весов
- Нормализация (BatchNorm, LayerNorm)

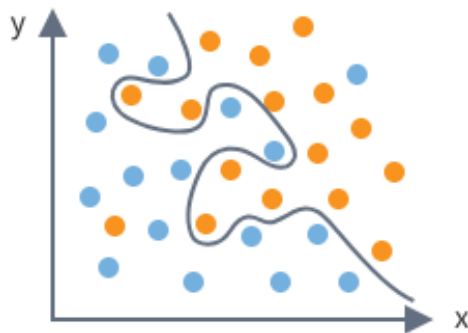
Регуляризация в нейронных сетях

Регуляризация — это набор приёмов, которые помогают предотвратить переобучение и сделать модель более устойчивой

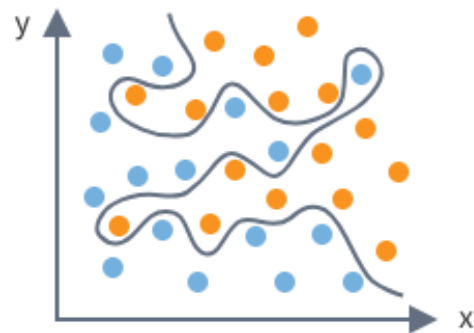
Переобучение нейросети (overfitting) — это ситуация, когда модель слишком точно подстраивается под обучающую выборку, запоминая её детали и шум, вместо того чтобы выявлять общие закономерности



Недообучение



Оптимум



Переобучение

Регуляризация в нейронных сетях

Без ограничений нейросеть может подогнаться под обучающие данные слишком сильно (запомнить их), и тогда она будет плохо работать на новых примерах.

Регуляризация добавляет к функции потерь дополнительный член, который наказывает модель за слишком большие или неустойчивые веса

Функция потерь с регуляризацией:

$$L_{total} = L_{data} + \lambda \cdot R(w)$$

L_{data} — исходная функция потерь (например, MSE для регрессии или кросс-энтропия для классификации)

$R(w)$ — регуляризатор, то есть «штраф» за сложность модели. Он зависит от весов w

λ — коэффициент, который контролирует баланс:

- Слишком маленькое значение λ : регуляризация почти не действует, сеть может переобучиться
- Слишком большое значение λ : сеть недообучается, потому что штраф «сильнее», чем данные

L1 (Lasso) регуляризация

L1 (Lasso) регуляризация — метод, при котором к функции ошибки добавляется штраф, зависящий от суммы абсолютных значений коэффициентов модели. Особенность этого подхода в том, что при достаточно сильной регуляризации некоторые веса могут становиться равными нулю. Таким образом, модель «отсекает» наименее значимые признаки и использует только действительно важные. Это делает Lasso удобным инструментом, когда нужно уменьшить количество признаков и одновременно провести их отбор (feature selection)

$$L_{total} = L_{data} + \lambda \cdot R(w)$$

$$R(w) = \sum_i |w_i|$$

L1 (Lasso) регуляризация

Как происходит зануление весов:

$$\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L_{\text{data}}}{\partial w_i} + \lambda \cdot \text{sign}(w_i)$$

Производная $|w_i|$ — это знаковая функция $\text{sign}(w_i)$

$$\text{sign}(w_i) = \begin{cases} +1, & w_i > 0 \\ -1, & w_i < 0 \\ 0, & w_i = 0 \end{cases}$$

$$w = w - \eta \left(\frac{\partial L_{\text{data}}}{\partial w} + \lambda \cdot \text{sign}(w) \right)$$

При L1-регуляризации маленькие коэффициенты постепенно уменьшаются, и как только сила регуляризации начинает доминировать над вкладом градиента ошибки, вес буквально прижимается к нулю и остаётся равным нулю в оптимальном решении.

L2 (Ridge) регуляризация

L2 (Ridge) регуляризация заключается в добавлении к функции потерь штрафного члена, пропорционального сумме квадратов коэффициентов модели.

В отличие от L1-регуляризации, данный метод не зануляет веса, а стремится уменьшить их величину, особенно сглаживая чрезмерно большие значения. Такой подход целесообразен в ситуациях, когда предполагается, что все признаки обладают информативностью и должны участвовать в модели, однако необходимо снизить их дисбаланс во влиянии на итоговый результат. Ridge-регуляризация способствует стабилизации решения, уменьшает дисперсию модели и предотвращает переобучение.

$$L_{total} = L_{data} + \lambda \cdot R(w)$$

$$R(w) = \sum_i w_i^2$$

L2 (Ridge) регуляризация

Функция потерь:

$$L_{\text{total}}(w) = L_{\text{data}}(w) + \lambda \sum_i w_i^2$$

Градиент:

$$\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L_{\text{data}}}{\partial w_i} + 2\lambda w_i$$

Обновление весов:

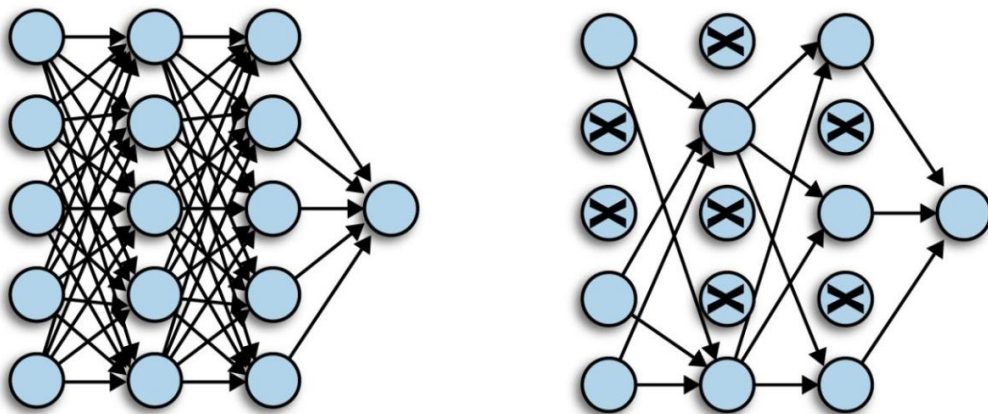
$$w_i = w_i - \eta \left(\frac{\partial L_{\text{data}}}{\partial w_i} + 2\lambda w_i \right)$$

При L2-регуляризации коэффициенты уменьшаются пропорционально своей величине: чем больше вес, тем сильнее на него действует штраф. В результате веса сглаживаются и стремятся к малым значениям, но не становятся строго равными нулю. Таким образом, модель сохраняет все признаки, но снижает их дисбаланс и устойчиво работает с новыми данными.

Dropout

Dropout — это метод регуляризации, используемый в нейронных сетях для снижения риска переобучения.

Его суть заключается в том, что во время обучения часть нейронов случайным образом временно отключается: их выходные значения принудительно приравниваются к нулю. Такое исключение выполняется независимо для каждого нейрона и на каждой итерации, благодаря чему сеть не привыкает полагаться на конкретные связи и учится строить более устойчивые представления



Dropout

У регуляризации Dropout есть гиперпараметр p , который задаёт вероятность «выключения» нейрона

Процесс обучения:

- Каждый нейрон вместе с его входящими и исходящими связями случайно исключается с вероятностью p
- На каждой итерации градиентного спуска отключается новый набор нейронов, поэтому веса обновляются только для оставшейся подсети

Формула для выхода одного нейрона:

$$y = f(Wx) \cdot m, \quad m \sim \text{Bernoulli}(1 - p)$$

m — случайная величина, принимающая значение 1 (нейрон работает) с вероятностью $1-p$ и 0 (нейрон выключен) с вероятностью p

$\text{Bernoulli}(1-p)$ — распределение Бернулли с параметром $1-p$. Это дискретное распределение: случайная величина может принимать только два значения — 0 или 1

Dropout

Inference (применение) нейронной сети, обученной с Dropout

При использовании обученной сети её структура уже фиксирована, поэтому нейроны не отключаются. Вместо этого компенсируют эффект Dropout масштабированием выходов на коэффициент $1-p$:

$$y = (1 - p) f(Wx)$$

Чтобы не изменять работу сети на этапе применения (inference), используют модифицированную версию Dropout: **Inverted Dropout**. В этом случае масштабирование выполняется уже во время обучения:

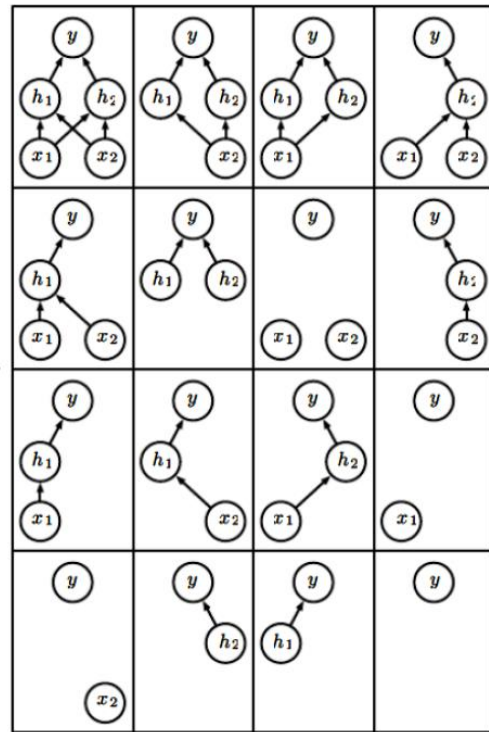
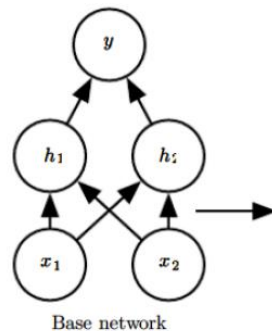
$$y = \frac{1}{1 - p} f(Wx) \cdot m, \quad m \sim \text{Bernoulli}(1 - p)$$

Благодаря такому приёму математическое ожидание выхода сохраняется на том же уровне, что и без Dropout. Поэтому при использовании обученной модели никаких дополнительных корректировок (например, умножения на $1-p$) уже не требуется

Dropout

Преимущества использования Dropout:

- Dropout — это метод регуляризации, применяемый при обучении нейронных сетей, основной эффект которого заключается в снижении переобучения
- Он препятствует явлению co-adaptation — ситуации, когда нейроны начинают чрезмерно подстраиваться друг под друга, что ухудшает способность модели к обобщению
- С точки зрения интерпретации, Dropout можно рассматривать как особую форму ансамблирования: на каждой итерации обучается случайная подсеть, но все они разделяют одни и те же параметры

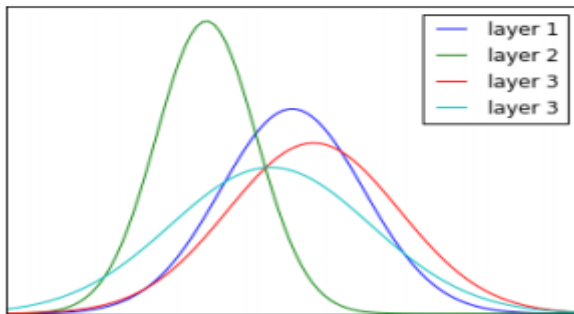


Batch normalization

При обучении глубоких нейронных сетей возникает проблема: изменения весов на одном слое (в процессе градиентного спуска) влияют на распределение входных данных для следующего слоя.

Изначально сигналы, поступающие на нейроны, имеют примерно нормальное распределение с некоторым средним и дисперсией. Такие распределения будут относительно стабильными и симметричными. Однако в ходе обучения, когда веса обновляются, распределение активаций меняется: может смещаться среднее значение и изменяться разброс

Это явление называется **internal covariate shift**. Оно замедляет процесс обучения и может приводить к тому, что сеть обучается нестабильно или не достигает оптимального качества.

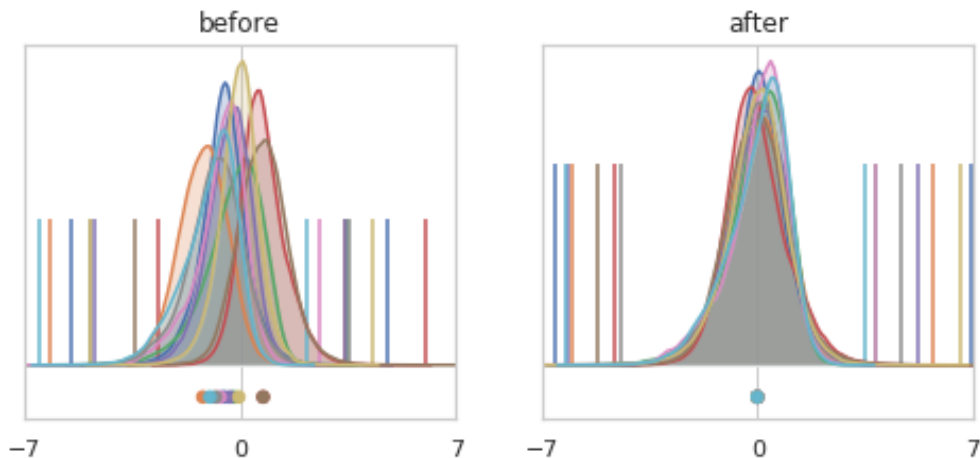


Batch normalization

Чтобы уменьшить влияние covariate shift на обучение нейросетей, можно использовать разные приёмы:

- очень тщательно подбирать начальные веса (что сложно и не всегда надёжно)
- уменьшать шаг градиентного спуска (что делает обучение крайне медленным)

Более эффективное решение — это **Batch Normalization (BatchNorm)**



Batch normalization

Пусть у нас есть мини-батч:

$$B = \{x_i\}_{i=1}^m$$

1. Вычисляем среднее по батчу:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

2. Считаем дисперсию по батчу:

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

3. Нормализуем значения:

$$x_i^{new} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Таким образом, активации приводятся к контролируемому распределению со средним ≈ 0 и дисперсией ≈ 1 . Это снимает зависимость распределения текущего слоя от изменений в предыдущих слоях

Batch normalization

Жёсткая нормализация (ноль и единица) может не всегда быть оптимальной. Поэтому в BatchNorm добавляются два обучаемых параметра:

$$y_i = \gamma x_i^{new} + \beta$$

γ — масштабирование

β — сдвиг

- Нормализация активаций стабилизирует процесс обучения, предотвращая резкие смещения распределений и неконтролируемый рост масштабов значений
- Обучаемые параметры γ и β возвращают модели гибкость, позволяя слоям адаптировать масштаб и сдвиг активаций под специфические требования задачи, вместо жёсткой фиксации в рамках нулевого среднего и единичной дисперсии.

Batch normalization

Преимущества использования Batch normalization:

- Повышает стабильность и ускоряет процесс обучения
- В ряде случаев делает использование Dropout избыточным
- За счёт ускоренного обучения облегчает тренировку более глубоких архитектур
- Снижает зависимость качества обучения от выбора начальной инициализации весов.

